

Assignment 4 - PIPES

//part 1- pipe

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>

#define max 30

int main(){
    int filedес[2], ch;
    char string[max];
    char readString[max];
    pid_t pid;
    printf("enter string to be given to parent: ");
    fgets(string, max, stdin);

    if(pipe(filedes) < 0){
        printf("pipe creation error!");
        exit(0);
    }
    pid = fork();
    if(pid < 0){
        printf("Fork creation was unsuccessful!");
        exit(0);
    }
```

```

else if(pid > 0){
    close(filedes[0]);
    write(filedes[1], string, max);
    close(filedes[1]);
    wait(NULL);
}

else if(pid == 0){
    close(filedes[1]);
    ch = read(filedes[0], readString, max);
    readString[ch] = '\0';
    printf("Data read by child is: %s", readString);
    close(filedes[0]);
}

return 0;
}

/*

```

OUTPUT

enter string to be given to parent: hello unnatti

Data read by child is: hello unnatti

```
*/
```

main.c	Run	Output
<pre> 1 //unnatti gogna 2 //part 1- pipe 3 4 #include<stdio.h> 5 #include<sys/types.h> 6 #include<unistd.h> 7 #include<stdlib.h> 8 #include<sys/wait.h> 9 #define max 30 10 int main(){ 11 int filedes[2], ch; 12 char string[max]; 13 char readString[max]; </pre>		<pre> enter string to be given to parent: hello unnatti Data read by child is: hello unnatti === Code Execution Successful === </pre>

//part 2- pipe

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>
#define max 30
int main(){
    int filed1[2],filed2[2],ch;
    char string1[max], string2[max];
    char readString[max], readString2[max];
    pid_t pid1, pid2;
    printf("enter string to be given to parent: ");
    fgets(string1, max, stdin);

    if(pipe(filed1) < 0){
        printf("pipe creation error!");
        exit(0);
    }
    pid1 = fork();
    if(pid1 < 0){
        printf("Fork creation was unsuccessful!");
        exit(0);
    }
    else if(pid1 > 0){
        close(filed1[0]);
        write(filed1[1], string1, max);
    }
```

```

        close(filedes1[1]);
        wait(NULL);
    }
    else if(pid1 == 0){
        close(filedes1[1]);
        ch = read(filedes1[0], readString, max);
        readString[ch] = '\0';
        printf("Data read by child of pipe 1 is: %s", readString);
        close(filedes1[0]);
    }

    pid2 = fork();
    printf("\nenter string to be given to child: ");
    fgets(string2, max, stdin);
    if(pipe(filedes2) < 0){
        printf("pipe creation error!");
        exit(0);
    }

    if(pid2 < 0){
        printf("Fork creation was unsuccessful!");
        exit(0);
    }

    else if(pid2 == 0 ){
        close(filedes2[0]);
        write(filedes2[1], string2, max);
        close(filedes2[1]);
        wait(NULL);
    }
}

```

```
else if(pid2 > 0){
    close(filedes2[1]);
    ch = read(filedes2[0], readString2, max);
    readString[ch] = '\0';
    printf("Data read by parent of pipe 2 is: %s", readString2);
    close(filedes2[0]);
}
return 0;

}
```